

livresgourmands.net

API REST · BD MySQL · JWT + bcrypt

Cours : Programmation Web avancée [420-WA6-AG] — Institut Grasset — Hiver 2026

Auteur : Arfaoui Yosri | Enseignant : Kahina TAMAZOUZT

1. Justification des choix du modèle de données

1.1 Normalisation (3NF)

Le schéma respecte la troisième forme normale (3NF). Chaque table possède une clé primaire entière auto-incrémentée. Toutes les dépendances fonctionnelles transitives ont été éliminées. Par exemple, les ouvrages référencent les catégories par FK plutôt que de dupliquer le nom. Le **prix_unitaire** est capturé au moment de l'ajout dans *panier_items* et *commande_items* pour préserver l'historique même si le prix d'un ouvrage change.

1.2 Choix des index

Table	Colonne(s)	Justification
ouvrages	categorie_id	Filtrage fréquent par catégorie
ouvrages	stock	WHERE stock > 0 sur toutes les listes publiques
commandes	client_id	Historique des commandes par client
avis	ouvrage_id	Calcul de la note moyenne par ouvrage
commentaires	ouvrage_id	Récupération des commentaires validés
listes_cadeaux	code_partage (UNIQUE)	Accès rapide par token de partage
avis	(client_id, ouvrage_id) UNIQUE	1 seul avis par client/ouvrage

1.3 Relations clés

Relation	Cardinalité	Description
users → panier	1 : N	Un client peut avoir plusieurs paniers (historique)
panier → panier_items	1 : N	Un panier contient plusieurs articles
users → commandes	1 : N	Un client peut passer plusieurs commandes
commandes → commande_items	1 : N	Une commande contient plusieurs lignes
ouvrages → avis	1 : N	Un ouvrage peut avoir plusieurs avis
users → listes_cadeaux	1 : N	Un client peut avoir plusieurs listes
listes_cadeaux → liste_items	1 : N	Une liste contient plusieurs ouvrages

2. Implémentation des règles métier

■ Filtrage stock > 0 (affichage public)

GET /api/ouvrages applique WHERE o.stock > 0 dans la requête SQL. Seuls les ouvrages disponibles sont retournés au public. GET /api/ouvrages/:id reste accessible même si stock=0 pour éviter les liens brisés.

■ Décrémentation transactionnelle du stock

POST /api/commandes utilise une transaction MySQL explicite (beginTransaction / commit / rollback). Pour chaque article du panier, l'API vérifie le stock disponible AVANT de décrémenter. Si un seul article est insuffisant, la transaction entière est annulée (ROLLBACK) et une erreur HTTP 400 est retournée avec le nom de l'ouvrage problématique.

■ Avis conditionnel à un achat

POST /api/ouvrages/:id/avis vérifie via une jointure SQL (commande_items JOIN commandes) que le client possède une commande avec statut='payee' contenant l'ouvrage. Si non, HTTP 403 est retourné. La contrainte UNIQUE(client_id, ouvrage_id) en base empêche les doublons (HTTP 409).

■ Modération des commentaires

À la soumission, valide est forcé à FALSE. Un éditeur ou admin appelle PUT /api/commentaires/:id/valider avec {"action":"valider"} ou {"action":"rejeter"}. La validation enregistre date_validation et valide_par. Seuls les commentaires valide=TRUE apparaissent dans le détail d'un ouvrage.

■ Paniers expirés (tâche cron)

Le script scripts/cleanPaniers.js exécute : UPDATE panier SET actif=FALSE WHERE actif=TRUE AND updated_at < DATE_SUB(NOW(), INTERVAL 24 HOUR). Il peut être planifié via cron (ex. tous les soirs à 2h) ou exécuté manuellement.

■ Protection des routes par rôles

Le middleware authenticate() vérifie le JWT. Le middleware authorize(...roles) vérifie que le rôle de l'utilisateur connecté est dans la liste autorisée. 4 rôles : client, editeur, gestionnaire, administrateur.

3. Limitations connues et améliorations possibles

Limitation	Amélioration proposée
Païement simulé	Intégration réelle Stripe/PayPal avec webhooks pour confirmer avant de décrémenter le stock
Pas de refresh token	Ajouter un refresh token JWT avec rotation et endpoint /api/auth/refresh
Pas de pagination	Ajouter ?page=&limit= avec LIMIT/OFFSET sur tous les endpoints de liste
Images des ouvrages	Ajouter un champ image_url ou une table ouvrage_images avec upload vers CDN (AWS S3)
Soft delete	Ajouter un champ deleted_at pour ne pas perdre l'historique des commandes
Tests automatisés	Ajouter Jest + Supertest pour des tests unitaires et d'intégration des endpoints

Rate limiting	Ajouter express-rate-limit pour protéger les routes /auth contre les attaques par force brute
----------------------	---